

P4CHECK: Verification and Debugging for Stateful P4 Programs

Tommaso Pacciani

t.c.pacciani@uva.nl

Informatics Institute

University of Amsterdam

Amsterdam, North Holland, The Netherlands

L. Thomas van Binsbergen

ltvanbinsbergen@acm.org

Informatics Institute

University of Amsterdam

Amsterdam, North Holland, The Netherlands

Chrysa Papagianni

c.papagianni@uva.nl

Informatics Institute

University of Amsterdam

Amsterdam, North Holland, The Netherlands

Abstract

Programmable data planes increasingly rely on stateful P4 programs to implement complex in-network functions. However, verifying and debugging these programs remains challenging because stateful temporal violations can span multiple packets and different program locations. We introduce P4CHECK, a tool that combines temporal verification with statement-level debugging for stateful P4 programs. P4CHECK uses a single definitional interpreter to execute P4 programs symbolically for verification and concretely for counterexample replay. Its specification language combines observations made while processing individual packets with temporal properties over packet sequences. When verification refutes a property, P4CHECK concretely replays the counterexample to support interactive debugging. We evaluate the fidelity of its executable semantics and its ability to detect bounded violations in representative real-world P4 programs.

CCS Concepts

• **Networks** → **Programming interfaces**; • **Software and its engineering** → **Formal software verification**; *Software testing and debugging*.

Keywords

P4, programmable data planes, formal verification, model checking, operational semantics, omniscient debugging

ACM Reference Format:

Tommaso Pacciani, L. Thomas van Binsbergen, and Chrysa Papagianni. 2026. P4CHECK: Verification and Debugging for Stateful P4 Programs. In *8th European P4 Workshop (EuroP4 '26)*, December 07–10, 2026, Utrecht, Netherlands. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3843192.3856163>



This work is licensed under a Creative Commons Attribution 4.0 International License.

EuroP4 '26, Utrecht, Netherlands

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2978-2/2026/12

<https://doi.org/10.1145/3843192.3856163>

1 Introduction

Modern networks increasingly rely on programmable data planes to implement packet-processing functions that were traditionally fixed in network-device hardware. P4 is a domain-specific language for specifying how programmable network devices parse, process, and forward packets, in order to meet specific service and operational requirements [6]. This flexibility enables functions such as in-band telemetry [20], intrusion detection [8, 9], and traffic classification [1] to be implemented directly in the data plane.

Much like conventional software, P4 programs can contain subtle logic errors that are difficult to detect. For general-purpose programming languages, developers can rely on testing frameworks, debuggers, static analysers, and formal verification tools, to identify and address such errors. Testing and formal verification both aim to increase confidence in program correctness, but they provide different guarantees. Testing evaluates program behavior over a finite set of selected inputs and execution scenarios, whereas formal verification can prove that a specified property holds across all inputs and executions captured by a formal model of the program and its environment. When the property does not hold, a verifier can return a counterexample, an input on which the program violates it.

After more than a decade of development, P4 has a rich ecosystem of developer tools, including testing and fuzzing frameworks [7, 12, 13, 29], simulators and emulators [23], and debuggers [27]. Numerous verification tools have also been developed for P4. The absence of general-purpose loops bounds program control flow, making automated verification tractable. Even so, architecture-defined externs and recirculation, which can trigger repeated pipeline execution, complicate faithful modeling of P4 programs.

Existing open-source P4 verification tools broadly fall into two groups. Stand-alone verifiers check built-in or user-supplied properties [10, 11, 15, 25, 28]. Their analyses range from language-level safety within individual pipelines to functional properties over complete device paths and temporal properties over packet sequences. The second group uses interactive theorem provers to formalize a broader portion

of P4's operational semantics, with some frameworks also supporting symbolic execution [2, 17, 26].

Across both groups, support for debugging property violations remains limited. For example, p4v [15] reconstructs execution traces for failed assertions, while Aquila [25] localizes faults by evaluating candidate program modifications. Neither, however, supports interactive exploration of a counterexample execution. This matters most for stateful programs. A temporal property constrains the behavior of a program over a sequence of packets, for example by requiring that a response is forwarded whenever its request was seen earlier. A counterexample to such a property spans several packets, and the events responsible for the violation can lie in different packets and at different program locations.

To address this gap, we introduce P4CHECK¹, a verification-to-debugging workflow for stateful P4 programs. P4CHECK's central novelty is bridging bounded temporal verification with statement-level debugging through a single, shared executable semantics of P4₁₆. Specifically, this paper makes the following contributions:

- **Integrated workflow:** We present a workflow that searches every packet sequence up to a fixed length for a violation of a temporal property and replays each counterexample in an omniscient debugger [21], which records the execution so that the developer can step backward as well as forward.
- **Dual-mode interpreter:** We give the semantics of P4₁₆ as a definitional interpreter [22], a Haskell program whose execution defines the meaning of each construct. The same interpreter runs symbolically for verification, with the packet and table entries left as unknowns [14], and concretely for replay, and it models pipeline semantics and externs across multiple architecture families.
- **Temporal navigation:** We provide a two-layer specification language for properties and an event-based breakpoint language that enables forward and backward navigation across statements and packet boundaries.
- **Empirical evaluation:** We validate P4CHECK against the reference p4c compiler test suite, packet tests generated by P4TestGen across 46 real-world programs, and representative temporal verification benchmarks.

The remainder of this paper is organized as follows. Section 2 introduces P4CHECK through a running example that illustrates its verification-to-debugging workflow. Section 3 presents the P4CHECK design, including its architecture, specification language, and breakpoint language. In Section 4 we evaluate its semantic conformance and verification performance. In Section 5 we discuss related work, and Section 6 concludes the paper.

¹P4CHECK is available open source at <https://github.com/TommasoPacciani/P4Check>.

```

1 cur_window.read(w, 0);
2 opened_at.read(opened, 0);
3 if (standard_metadata.ingress_global_timestamp - opened >= T) {
4     w = w + 1;
5     cur_window.write(0, w);
6     opened_at.write(0, standard_metadata.ingress_global_timestamp);
7 }
8
9 if (meta.is_request == 1) {
10     seen.write(meta.flow_index, w + 1);
11     standard_metadata.egress_spec = EXTERNAL_PORT;
12 } else if (meta.is_response == 1) {
13     seen.read(s, meta.flow_index);
14     if (s == w + 1) {
15         standard_metadata.egress_spec = INTERNAL_PORT;
16     } else {
17         mark_to_drop(standard_metadata);
18     }
19 }

```

Figure 1: The causal core of the response-window firewall. A request writes the current window stamp, while a response is forwarded only if its stored stamp still matches. Packet classification and bidirectional hashing are omitted.

2 P4Check by Example

P4CHECK takes a P4 program and a property. It searches every packet sequence up to a fixed length for one on which the program violates the property. If it finds one, it reports the sequence as a counterexample, together with the timestamps, table entries, and register contents the violation depends on, and replays it in the debugger. We demonstrate P4CHECK's verification-to-debugging workflow using a stateful response-window firewall written in P4₁₆ for the v1model architecture, inspired by [26]. The firewall admits an external response packet only if a corresponding internal request was observed within the preceding T time units. Packets arriving on internal port 1 are requests, while those arriving on external port 2 are responses. Each packet's TCP five-tuple is hashed symmetrically to an index in a 256-cell register `seen`. Two 1-element registers maintain a coarse-grained, packet-driven time window: `cur_window` stores the active window identifier w , and `opened_at` stores the timestamp `opened` when that window started. When any packet arrives whose ingress timestamp (`standard_metadata.ingress_global_timestamp`) is at least T time units beyond `opened`, the program advances w and updates `opened_at`. When an internal request arrives, it stamps its flow's cell in `seen` with $w + 1$; the offset by one distinguishes an active stamp from the register's initial value of zero. When an external response arrives, it reads the stored stamp s from `seen` and is admitted only when $s == w + 1$. Figure 1 shows the core logic of this program.

$$\begin{aligned}
& \forall i : \text{bit}(32). \forall t_0 : \text{bit}(48). \\
& G \left(\text{meta.flow_index} = i \wedge \text{meta.is_request} = 1 \right. \\
& \quad \wedge \text{standard_metadata.ingress_global_timestamp} = t_0 \\
& \quad \rightarrow G \left(\text{meta.flow_index} = i \wedge \text{meta.is_response} = 1 \right. \\
& \quad \quad \wedge \text{standard_metadata.ingress_global_timestamp} - t_0 \leq T \\
& \quad \quad \left. \rightarrow \neg \text{drop} \right) \left. \right)
\end{aligned}$$

Figure 2: The response-window property. A response mapped to the request’s cell and arriving no more than T time units later must not be dropped.

Table 1: Three-packet counterexample refuting the response-window property ($T = 10^6$).

Epoch	Type / Flow	Time	Window (w)	Stamp (s)	Result
0	Request, flow i	$T - 1$	0	1	Forwarded
1	Request, flow j	T	$0 \rightarrow 1$	2	Forwarded
2	Response, flow i	$T + 1$	1	1	Dropped ($s \neq w + 1$)

Temporal property. P4CHECK models each completed packet execution as one epoch. Figure 2 specifies the response-window policy over packet epochs. Expressions in P4CHECK are sampled at named program locations; in this example, all expressions are observed at the end of ingress (`MyIngress.end`), with explicit location annotations omitted for brevity. The quantified cell index i and request arrival time t_0 are bound when a request is observed and remain fixed as temporal operators evaluate subsequent epochs.

The outer G operator quantifies over every epoch in which a request is processed. When a request matches index i at timestamp t_0 , the inner G checks all subsequent epochs for a response mapped to the same index i arriving within T time units. The inner G is necessary because arbitrary packets from other flows may interleave between a request and its response.

Counterexample. P4CHECK refutes the property with a three-packet counterexample, summarized in Table 1. Initially, all register cells hold zero. At epoch 0 ($t = T - 1$), an internal request for flow index i arrives, stores the stamp $s = w + 1 = 1$, and is forwarded. At epoch 1 ($t = T$), an unrelated internal request for index j arrives; because T time units have elapsed since window 0 opened, it advances the global window w from 0 to 1. At epoch 2 ($t = T + 1$), an external response for index i arrives, only two time units after its request. It reads its flow’s stamp $s = 1$, but the admission test expects $s == w + 1 = 2$, causing the firewall to drop a valid, timely response.

Replay and diagnosis. P4CHECK concretizes this symbolic counterexample and replays it through its concrete interpreter, recording an execution trace for the debugger. The replay faithfully reproduces the drop at epoch 2, where the debugger initially lands the developer. Inspecting the execution context confirms that $s = 1$ and $w = 1$, so the condition $s == w + 1$ fails. To locate where w was set, the developer can navigate backward across the packet boundary using a temporal breakpoint, `before(when(cur_window.write))`. This lands directly at epoch 1, where an unrelated packet at index j advanced the shared global window w from 0 to 1. Because the global window counter is shared across all flows, advancing w effectively invalidated flow i ’s stamp just one time unit after it was written, without ever touching cell i . The response at epoch 2 exposes the failure, but the preceding packet at epoch 1 caused it.

Repair. The check in Figure 1 admits a response only when its stored stamp satisfies $s == w + 1$. A stamp recorded in window w holds the value $w + 1$. When an unrelated packet triggers a rotation, the global counter increments to $w + 1$. As a result, the previously recorded stamp now equals the current value of w . To prevent premature eviction, the repair also admits a response if its stamp matches $s == w$, provided the stamp is nonzero:

```

if (s != 0 && (s == w + 1 || s == w)) {
    standard_metadata.egress_spec = INTERNAL_PORT;
}

```

A stamp now remains valid across two consecutive windows. Each rotation updates `opened_at` and requires at least T elapsed time units before the next rotation can occur. Consecutive rotations are therefore separated by at least T time units. Consequently, every request remains authorized for at least T time units regardless of when it arrived within its window. The guard $s != 0$ prevents admitting uninitialized flows. Before the first rotation, both an unrecorded cell and the window counter w are zero. Without the nonzero guard, unsolicited external packets would match $s == w$ immediately.

After applying this repair, rerunning P4CHECK on the repaired program reveals no counterexample within the configured packet bound. While the repair guarantees that requests remain valid for at least T time units, a request may remain authorized longer because window rotations are driven solely by packet arrivals. Enforcing an exact upper bound on authorization would require per-cell timestamps instead of a shared global window.

3 P4CHECK System Design

Architecture. Figure 3 shows how P4CHECK connects verification with debugging. At its center is a definitional P4 interpreter implemented in Haskell. The interpreter uses

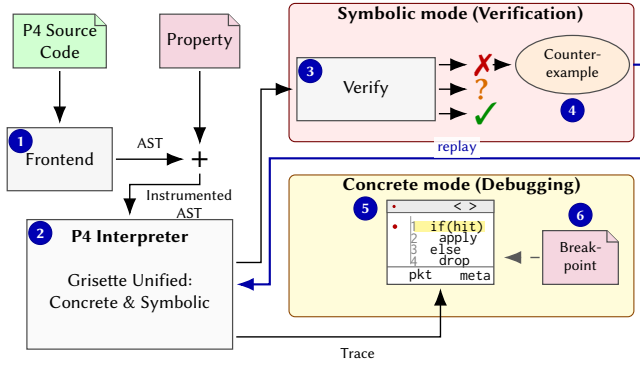


Figure 3: P4CHECK checks temporal properties by symbolically interpreting a P4 program. When a property is refuted, the same interpreter replays the concrete counterexample and records the execution trace presented in the debugger.

Grisette [16], a symbolic-execution library that provides an API for writing interpreters that can run in either concrete or symbolic mode. P4CHECK uses concrete execution for counterexample replay and symbolic execution for verification.

(1) Frontend and instrumentation. Given a P4 program and property, the frontend parses and type-checks the P4 source to produce an abstract syntax tree (AST). It then instruments the AST with ghost variables that capture the runtime observations referenced by the property.

(2) Symbolic execution. The Grisette-based interpreter executes the instrumented AST in symbolic mode. In this mode the packet, its metadata, and the table entries are symbolic, that is, they stand for unknown values. A single run therefore covers every path through the program and produces a formula that relates these symbolic values to the observations of the property.

(3) Verification. Symbolic execution characterizes the processing of one packet, and model checking links repeated symbolic executions through persistent state to determine whether a packet sequence violates the property. P4CHECK uses bounded model checking [5], which considers packet sequences up to a fixed length. The Z3 SMT solver decides whether some assignment to the symbolic values of such a sequence violates the property. A satisfying assignment is a counterexample. If there is none, the property holds for every sequence within the bound, which does not prove it for longer ones.

For most constructs, the symbolic and concrete modes are provided by a single interpreter using the Grisette API. In a few cases, P4CHECK uses overapproximations, allowing the symbolic model to admit behaviors beyond those strictly realizable by concrete execution. For example, complex hash functions cannot be efficiently evaluated symbolically, so the

verifier overapproximates them as unconstrained symbolic variables. To ensure that an overapproximation does not result in spurious counterexamples, we use the concrete interpreter to validate the counterexample.

(4) Counterexample generation. When generating a counterexample, overapproximations such as table choices are converted into inputs the concrete interpreter can consume. For example, a symbolic table-action choice is translated into concrete control-plane entries that reproduce the same behavior during replay. The counterexample also contains the packet sequence, required architecture-specific inputs such as timestamps, and any other assignments needed to reproduce the violation. For example, an uninitialized register may allow an arbitrary value to be read. If the violation depends on this free choice, the counterexample must retain the chosen value so that concrete replay can reproduce it accurately.

(5) Replay and execution trace generation. The interpreter replays the counterexample in concrete mode and records a source-level execution trace. The debugger operates over this trace and allows the developer to step backward and forward through the execution and inspect values at any point.

(6) Debugging. Because multi-packet execution traces can contain many fine-grained statement-level events, manual stepping can be tedious. P4CHECK therefore provides a break-point language for navigating the trace to relevant program points. The debugger automatically pre-populates break-points targeting the observations in the verified property, while also letting developers write custom queries.

3.1 Specification Language

P4Check’s specification language has two layers. Intra-packet observations describe values, control-flow events, packet inputs, and packet outcomes within a single packet-processing epoch. Inter-packet Linear Temporal Logic (LTL) [4] formulas relate those observations across epochs to express temporal properties over packet sequences. Figure 4 presents the resulting language.

Intra-packet observations	Inter-packet formulas
$loc ::= q.begin \mid q.end$	$\varphi ::= atom \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2$
$o ::= e@loc \mid hit@q \mid drop$	$\mid X\varphi \mid \varphi_1 \cup \varphi_2$
$atom ::= o \mid const \mid \neg atom \mid atom_1 \oplus atom_2$	$\mid \forall x. \varphi$
$\mid if\ atom\ then\ atom\ [else\ atom]$	

Figure 4: A shared intra-packet observation language and its epoch-level LTL composition.

The metavariable o ranges over primitive observations, while $atom$ ranges over typed expressions that combine their values. The observation $e@loc$ evaluates the P4 expression e

at the selected boundary. Here q denotes a program, control, table, action, or other named semantic block, while `begin` and `end` refer to its entry and exit points.

The observation $hit@q$ is Boolean and records whether execution reaches the block. Similarly, the Boolean observation $drop$ indicates that packet processing produces no output packet, while $port_{in}$ and $port_{out}$ expose the packet's ingress and egress ports. Constants, conditionals, and the operators represented by $\oplus$ form typed expressions over these observed values; only Boolean expressions may occur as atomic propositions of an LTL formula.

Each epoch supplies one Boolean valuation for the atomic propositions in an LTL formula. The X and U operators therefore advance between packet-processing epochs rather than between events within an epoch. $X\varphi$ holds at an epoch if φ holds at the next one. $\varphi_1 U \varphi_2$ holds if φ_2 holds at the current or a later epoch and φ_1 holds at every epoch before it. Implication and G are derived operators. $G\varphi$, read as globally φ , holds when φ holds at the current epoch and at every later one. Persistent P4 state, such as register contents, carries information from one epoch to the next. Finally, the quantifier $\forall x$. φ introduces a symbolic P4 value that is fixed across the complete execution, allowing a property, for example, to refer to an arbitrary register index.

Breakpoint language. The debugger represents a replay as source-level events that record entry to or exit from a P4 construct together with the concrete program state. Event predicates combine P4 expressions with source locations. The breakpoint `when  $p$` stops at the first event satisfying p . The breakpoint p_1 `until  $p_2$` returns the event where p_1 first becomes true if it remains true until a later event satisfies p_2 . The operators `after` and `before` restrict where the debugger searches for another breakpoint. Unlike the epoch-level temporal operators in a verification specification, these operators advance between statement-level events and can navigate within one packet or across packet boundaries.

4 Evaluation

In the following we evaluate P4CHECK along two axes: (1) the fidelity of its concrete execution semantics and (2) its verification performance against P4TV [28] on stateful benchmarks.

4.1 Semantic validation

We evaluate P4CHECK's concrete execution semantics against two independent sources of expected behavior. First, P4CHECK executes the runtime test cases of p4c, the reference P4 compiler [19]. Each case pairs a P4 program with an STF (Simple Test Framework) script that feeds input packets and records the expected output packets. P4CHECK passes 202 of the 262 runtime cases (77.1%). Of the remaining cases, 35 stop before

execution because of unsupported P4 constructs or type-checking failures, one uses an unsupported STF construct, and 24 complete execution but produce an incorrect output. Among the 226 cases for which an output can be compared, P4CHECK matches the expected output in 202 cases (89.4%).

The p4c cases test the compiler. They cover the core language but not all externs, such as checksums and cloning. We therefore also use P4TestGen [24] to generate input-output pairs for feasible execution paths through real-world P4 programs. With the STF backend and a limit of 150 cases per program, the recorded evaluation contains 3,156 cases: all 3,002 generated cases for 44 programs and 154 cases from two additional programs. P4CHECK matches P4TestGen's expected output in 1,402 of the 1,691 cases that reach output comparison (82.9%). It matches every generated case for 11 of the 44 fully evaluated programs. Of the other recorded cases, 1,465 stop before output comparison because parsing or compilation fails or the case exercises an unsupported feature, including deprecated primitives and the v1model log-message extern. In the remaining 289 cases, execution completes, but P4CHECK's output disagrees with P4TestGen's expected output.

These results show that P4CHECK agrees with the reference output in most cases that reach execution, including complete agreement for 11 real-world programs. The 24 p4c mismatches and 289 P4TestGen mismatches nevertheless show that its executable semantics does not yet cover all expected P4 behavior.

4.2 Verification results and performance

We compare P4CHECK with P4TV [28], the only other tool that checks temporal properties of P4 programs. Table 3 reports the time each tool needs on eight cases. Each case pairs a P4 program with a temporal property that does not hold. Four cases come from P4TV's evaluation unchanged. For the other four we took a program from that evaluation whose property holds and injected a bug by changing one statement. We left out the two largest programs, Blink and NDN. We compare performance only on finding violations, because bounded model checking proves a property by exhaustion, which is too costly for these programs. For each case, both tools receive the same P4 program and property. We ran both on an AMD Ryzen 9 PRO 8945HS with 64 GB of memory under Ubuntu 24.04, with Z3 4.16.0 for P4CHECK and P4TV in a Docker container. The property is written in P4TV's LTL input language, and P4CHECK translates it into its specification language (Section 3.1). The translation covers every construct used by the benchmark properties, but not all of P4LTL. P4CHECK searches packet sequences of increasing length and stops at the first counterexample, whereas P4TV performs unbounded temporal verification.

Tool	Verification scope		Diagnosis			Breadth	
	Stateful P4	Temporal specs.	Concrete CEX	Auto-localize	CEX exploration	Multi-arch.	Open
ASSERT-P4 [11, 18]	~					~	✓
p4v [15]	✓		✓				
bf4 [10]				✓		~	✓
Aquila [25]	✓		✓	✓			
P4TV [28]	✓	✓	✓			~	✓
HOL4P4 [2, 17]	✓					✓	✓
P4Check	✓	✓	✓		✓	✓	✓

Table 2: Capabilities of selected P4 verification tools. A check mark denotes support, ~ partial support, and an empty cell no support. CEX stands for counterexample.

Table 3: Time to find a counterexample. Row numbers refer to Table 2 of the P4TV paper [28]. An asterisk marks an injected bug.

Program	P4TV row	P4Check	P4TV
BFS	1*	10.9 s	59.5 s
P4-CoDel	4*	12.4 s	215.7 s
DFS	5*	9.2 s	37.7 s
P4NIS	9	11.2 s	137.7 s
P4sp	13	7.2 s	73.5 s
P4sp	15	7.4 s	77.3 s
P4xos acceptor	18	29.9 s	178.0 s
P4xos learner	19*	25.6 s	timeout (240 s)

P4CHECK finds a counterexample in all eight cases, in 7.2 to 29.9 seconds. P4TV reports the violation in seven cases and needs between 37.7 and 215.7 seconds, so P4CHECK shows a speedup of 4.1× to 17.4× over P4TV. On the P4xos learner case P4TV reaches the timeout of 240 seconds. This is unexpected, since it proves the same property on the original program in 21.5 seconds.

Every counterexample in Table 3 has only two packets, which is a best case for a bounded search. We therefore also experimented with a program whose property is first violated at packet N . By varying N we found that P4CHECK’s time grows with N , and that from about ten packets on P4TV is faster.

5 Related Work

Table 2 summarizes the verification scope, diagnosis support and architecture coverage of existing P4 verification tools.

bf4 [10] checks safety properties that apply to any P4 program, such as invalid header accesses and out-of-bounds register accesses. ASSERT-P4 [11, 18], p4v [15] and Aquila [25] check user-specified assertions about the processing of a

single packet, and Aquila also localizes the program parts that contribute to a failure. HOL4P4 [3, 17] and Verifiable P4 [26] formalize the semantics of P4 in interactive theorem provers. P4TestGen [24] generates input-output tests for the control-flow paths of a P4 program.

P4TV [28] is the only other tool that verifies temporal properties over packet sequences. It can refer to header, metadata, and register values only at packet-processing boundaries, and it fully supports only the v1model architecture. P4CHECK instead observes P4 expressions at named program locations, supports multiple architecture families, and provides a debugger to inspect counterexample executions across statements and packet boundaries.

6 Conclusion

P4CHECK connects bounded temporal verification with concrete replay and statement-level debugging through a shared executable P4 semantics. In the response-window example, P4CHECK concretizes a symbolic counterexample into an execution trace, allowing the developer to follow the dropped response backward to the preceding packet’s causal state update. P4CHECK agrees with most reference outputs in cases that reach output comparison, but its semantics still produces output mismatches. On the benchmark cases, whose counterexamples have two packets, it is 4.1× to 17.4× faster than P4TV. In a separate experiment we see that P4CHECK’s time grows with the length of the counterexample, and that P4TV is faster on long counterexamples. The current prototype therefore does not match P4TV’s performance in general. To close this gap, future work will add the ability to call external model checkers, like P4TV does. We will also broaden semantic coverage. By connecting temporal property violations to the program events that caused them, P4CHECK helps developers move from detecting failures to understanding their causes.

References

- [1] Aristide Tanyi-Jong Akem, Guillaume Fraysse, and Marco Fiore. 2024. Encrypted Traffic Classification at Line Rate in Programmable Switches with Machine Learning. In *NOMS 2024-2024 IEEE Network Operations and Management Symposium*. 1–9. doi:10.1109/NOMS59830.2024.10575394
- [2] Anoud Alshnakat, Didrik Lundberg, Roberto Guanciale, and Mads Dam. 2024. HOL4P4: Mechanized Small-Step Semantics for P4. *Proc. ACM Program. Lang.* 8, OOPSLA1, Article 102 (April 2024), 27 pages. doi:10.1145/3649819
- [3] Anoud Alshnakat, Didrik Lundberg, Roberto Guanciale, Mads Dam, and Karl Palmkog. 2022. HOL4P4: semantics for a verified data plane. In *Proceedings of the 5th International Workshop on P4 in Europe (Rome, Italy) (EuroP4 '22)*. Association for Computing Machinery, New York, NY, USA, 39–45. doi:10.1145/3565475.3569081
- [4] Christel Baier and Joost-Pieter Katoen. 2008. *Principles of Model Checking*. The MIT Press.
- [5] A. Biere, A. Cimatti, E.M. Clarke, O. Strichman, and Y. Zhu. 2003. Bounded Model Checking. *Advances in Computers* 58 (2003), 117–148. doi:10.1016/S0065-2458(03)58003-2
- [6] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. 2014. P4: programming protocol-independent packet processors. *SIGCOMM Comput. Commun. Rev.* 44, 3 (July 2014), 87–95. doi:10.1145/2656877.2656890
- [7] Pietro Bressana, Noa Zilberman, and Robert Soulé. 2020. Finding hard-to-find data plane bugs with a PTA. In *Proceedings of the 16th International Conference on Emerging Networking EXperiments and Technologies (Barcelona, Spain) (CoNEXT '20)*. Association for Computing Machinery, New York, NY, USA, 218–231. doi:10.1145/3386367.3431313
- [8] Yaying Chen, Siamak Layeghy, Liam Daly Manocchio, and Marius Portmann. 2024. P4-NIDS: High-Performance Network Monitoring and Intrusion Detection in P4. arXiv:2411.17987 [cs.NI] <https://arxiv.org/abs/2411.17987>
- [9] Roberto Doriguzzi-Corin, Luis Augusto Dias Knob, Luca Mendozzi, Domenico Siracusa, and Marco Savi. 2024. Introducing Packet-Level Analysis in Programmable Data Planes to Advance Network Intrusion Detection. arXiv:2307.05936 [cs.CR] <https://arxiv.org/abs/2307.05936>
- [10] Dragos Dumitrescu, Radu Stoescu, Lorina Negreanu, and Costin Raiciu. 2020. bf4: towards bug-free P4 programs. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication (Virtual Event, USA) (SIGCOMM '20)*. Association for Computing Machinery, New York, NY, USA, 571–585. doi:10.1145/3387514.3405888
- [11] Lucas Freire, Miguel Neves, Lucas Leal, Kirill Levchenko, Alberto Schaeffer-Filho, and Marinho Barcellos. 2018. Uncovering Bugs in P4 Programs with Assertion-based Verification. In *Proceedings of the Symposium on SDN Research (Los Angeles, CA, USA) (SOSR '18)*. Association for Computing Machinery, New York, NY, USA, Article 4, 7 pages. doi:10.1145/3185467.3185499
- [12] Qiao Kang, Jiarong Xing, Yiming Qiu, and Ang Chen. 2021. Probabilistic profiling of stateful data planes for adversarial testing. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (Virtual, USA) (ASPLOS '21)*. Association for Computing Machinery, New York, NY, USA, 286–301. doi:10.1145/3445814.3446764
- [13] Jiwon Kim, Dave Jing Tian, and Benjamin E. Ujcich. 2025. Chimera: Fuzzing P4 Network Infrastructure for Multi-Plane Bug Detection and Vulnerability Discovery. In *2025 IEEE Symposium on Security and Privacy (SP)*. 3088–3106. doi:10.1109/SP61157.2025.00194
- [14] James C. King. 1976. Symbolic execution and program testing. *Commun. ACM* 19, 7 (July 1976), 385–394. doi:10.1145/360248.360252
- [15] Jed Liu, William Hallahan, Cole Schlesinger, Milad Sharif, Jeongkeun Lee, Robert Soulé, Han Wang, Călin Cașcaval, Nick McKeown, and Nate Foster. 2018. p4v: practical verification for programmable data planes. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication (Budapest, Hungary) (SIGCOMM '18)*. Association for Computing Machinery, New York, NY, USA, 490–503. doi:10.1145/3230543.3230582
- [16] Sirui Lu and Rastislav Bodik. 2023. Griset: Symbolic Compilation as a Functional Programming Library. *Proc. ACM Program. Lang.* 7, POPL, Article 16 (Jan. 2023), 33 pages. doi:10.1145/3571209
- [17] Didrik Lundberg, Roberto Guanciale, and Mads Dam. 2024. Proof-Producing Symbolic Execution for P4. In *Verified Software. Theories, Tools and Experiments: 16th International Conference, VSTTE 2024, Prague, Czech Republic, October 14–15, 2024, Revised Selected Papers (Prague, Czech Republic)*. Springer-Verlag, Berlin, Heidelberg, 70–83. doi:10.1007/978-3-031-86695-1_5
- [18] Miguel Neves, Lucas Freire, Alberto Schaeffer-Filho, and Marinho Barcellos. 2018. Verification of P4 programs in feasible time using assertions. In *Proceedings of the 14th International Conference on Emerging Networking EXperiments and Technologies (Heraklion, Greece) (CoNEXT '18)*. Association for Computing Machinery, New York, NY, USA, 73–85. doi:10.1145/3281411.3281421
- [19] P4 Language Consortium. [n. d.]. p4c: The Reference Compiler for the P4 Programming Language. <https://github.com/p4lang/p4c>. Accessed: 2026-08-13.
- [20] Konstantinos Papadopoulos, Panagiotis Papadimitriou, and Chrysa Papagianni. 2023. Deterministic and Probabilistic P4-Enabled Lightweight In-Band Network Telemetry. *IEEE Trans. on Netw. and Serv. Manag.* 20, 4 (Dec. 2023), 4909–4922. doi:10.1109/TNSM.2023.3301839
- [21] Guillaume Pothier and Éric Tanter. 2009. Back to the Future: Omniscient Debugging. *IEEE Softw.* 26, 6 (Nov. 2009), 78–85. doi:10.1109/MS.2009.169
- [22] John C. Reynolds. 1998. Definitional Interpreters Revisited. *Higher Order Symbol. Comput.* 11, 4 (Dec. 1998), 355–361. doi:10.1023/A:1010075320153
- [23] Fabricio Rodriguez, Francisco Germano Vogt, Ariel Góes De Castro, Marcos Felipe Schwarz, and Christian Rothenberg. 2022. P4 programmable patch panel (P7): an instant 100g emulated network on your tofino-based pizza box. In *Proceedings of the SIGCOMM '22 Poster and Demo Sessions (Amsterdam, Netherlands) (SIGCOMM '22)*. Association for Computing Machinery, New York, NY, USA, 4–6. doi:10.1145/3546037.3546046
- [24] Fabian Ruffy, Jed Liu, Prathima Kotikalapudi, Vojtech Havel, Hanneli Tavante, Rob Sherwood, Vladyslav Dubina, Volodymyr Peschenko, Anirudh Sivaraman, and Nate Foster. 2023. P4Testgen: An Extensible Test Oracle For P4. In *Proceedings of the ACM SIGCOMM 2023 Conference (New York, NY, USA) (ACM SIGCOMM '23)*. Association for Computing Machinery, New York, NY, USA, 136–151. doi:10.1145/3603269.3604834
- [25] Bingchuan Tian, Jiaqi Gao, Mengqi Liu, Ennan Zhai, Yanqing Chen, Yu Zhou, Li Dai, Feng Yan, Mengjing Ma, Ming Tang, Jie Lu, Xionglie Wei, Hongqiang Harry Liu, Ming Zhang, Chen Tian, and Minlan Yu. 2021. Aquila: a practically usable verification system for production-scale programmable data planes. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference (Virtual Event, USA) (SIGCOMM '21)*. Association for Computing Machinery, New York, NY, USA, 17–32. doi:10.1145/3452296.3472937
- [26] Qinshi Wang, Mengying Pan, Shengyi Wang, Ryan Doenges, Lennart Beringer, and Andrew W. Appel. 2023. Foundational Verification of

- Stateful P4 Packet Processing. In *14th International Conference on Interactive Theorem Proving (ITP 2023) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 268)*, Adam Naumowicz and René Thiemann (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 32:1–32:20. doi:10.4230/LIPIcs.ITP.2023.32
- [27] Cheng Zhang, Jun Bi, Yu Zhou, Jianping Wu, Bingyang Liu, Zhaogeng Li, Abdul Basit Dogar, and Yangyang Wang. 2017. P4DB: On-the-fly debugging of the programmable data plane. In *2017 IEEE 25th International Conference on Network Protocols (ICNP)*. 1–10. doi:10.1109/ICNP.2017.8117548
- [28] Delong Zhang, Chong Ye, and Fei He. 2025. On temporal verification of stateful P4 programs. In *Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation (Philadelphia, PA, USA) (NSDI '25)*. USENIX Association, USA, Article 13, 17 pages.
- [29] Naiqian Zheng, Mengqi Liu, Ennan Zhai, Hongqiang Harry Liu, Yifan Li, Kaicheng Yang, Xuanzhe Liu, and Xin Jin. 2022. Meissa: scalable network testing for programmable data planes. In *Proceedings of the ACM SIGCOMM 2022 Conference (Amsterdam, Netherlands) (SIGCOMM '22)*. Association for Computing Machinery, New York, NY, USA, 350–364. doi:10.1145/3544216.3544247